Database Systems Implementation
Mini-Project

1075201

Computer Science (Part C)

Hilary 2026

1

a

i & ii

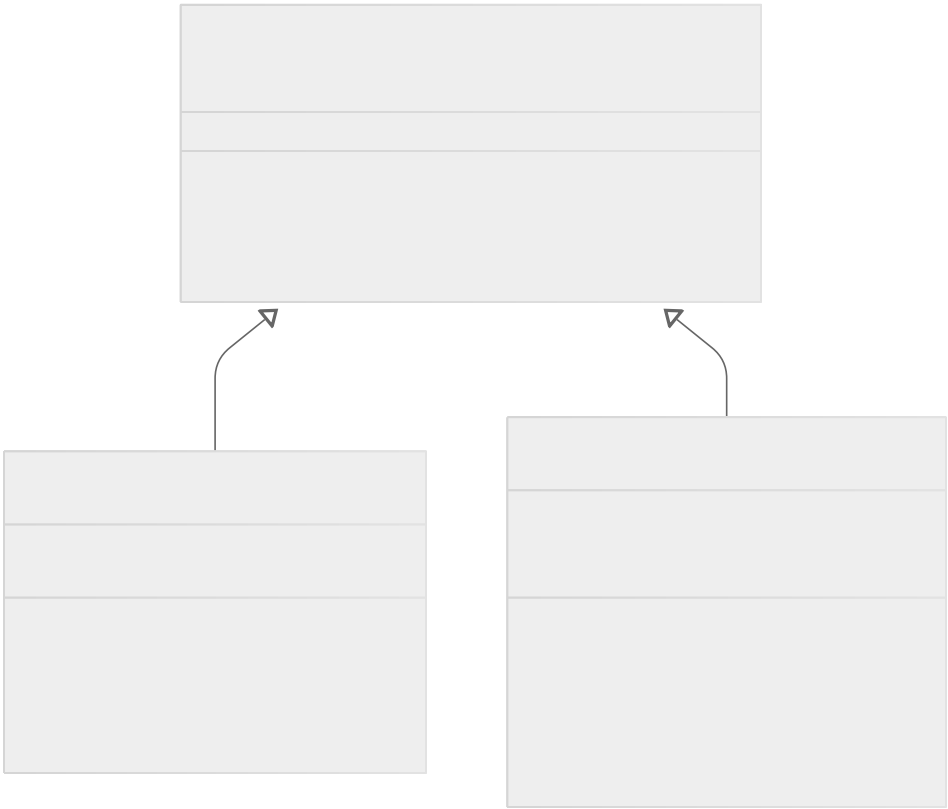

Figure 1: Lock Hierarchy

We implement two different types of locks: shared locks (SLocks) and shared/exclusive locks (XSLocks). SLocks allow multiple transactions to hold the lock simultaneously, they are used for predicate locks and are acquired during query evaluation. XSLocks, on the other hand, are the typical locks seen in normal two-phase locking: they can operate in shared mode or exclusive mode, and are used for tuple level locking during both edits (add/delete) and queries.

Within each Lock (XSLock and SLock), we maintain an explicit set of transactions holding the lock. This will later be used by the deadlock detector to find out which transactions are being held by which locks, which is necessary in traversing the implicit waits-for graph. We choose to explicitly use inheritance and a VTable to implement the lock hierarchy, as opposed to using a C++20 concept. This allows each transaction to hold onto sets of Lock pointers, allowing them to be treated uniformly.

The set of lock_holders is implemented as a std::flat_set since we expect the number of transactions holding a lock to be small, and we want to optimize for cache locality.

XSLocks are directly incorporated into the DataTuple class (Figure 2), which represents a single tuple within a relation.

Figure 2: DataTuple Class

The Slocks are used in two ways: for a whole relation predicate lock, and for a group predicate lock (Figure 3)

Figure 3: Relation and Group Classes

Groups are used as both indices as well as allocated objects that contain the relevant SLock, with the semantics that the SLock should cover all tuples in the group. We have three main types of groups: left-constant groups, right-constant groups, and diagonal groups (where left == right). With this, we are able to effectively cover predicates of the form R(c, x), R(x, c), and R(x, x) respectively, where c is some constant value and x is a variable. We will later see in 1(c) how these are the natural groupings for the types of queries we have to evaluate.

We also have a whole relation lock that is used to cover predicates of the form R(x, y), i.e. that covers the entire relation. It is possible to have made this into a group too, but we choose to keep it separate since we already have enough indices to effectively cover the queries.

Predicate locks are meant to prevent phantom rows, which are rows that are not present during the initial read of a transaction, but appear later due to concurrent transactions. As only queries look over "ranges" of tuples, they are the only operations that need to acquire predicate locks. Edits, on the other hand, only need to acquire tuple-level locks since it is explicitly specified which tuples they are adding/deleting.

For a predicate lock to prevent a phantom, edits need to check that the relevant predicate locks covering the tuple they are about to edit do "permit" the edit, i.e. that they are not currently held by another transaction.

If a query acquires a predicate lock that covers a tuple, it doesn't need to acquire the tuple-level lock for that tuple, since other edits will be stopped by the predicate lock before they can even acquire the tuple-level lock. However, to safely prevent dirty reads, queries still need to check that the tuple-level lock permits the read, i.e. that it is not currently held in exclusive mode by another transaction.

Note that it is possible for a query to not acquire any predicate lock that covers a tuple it is about to read, in which case it needs to acquire the tuple-level lock in shared mode to prevent concurrent edits.

Apart from belonging to relations, groups or tuples, pointers to locks are held by transactions (Figure 4). We track the locks held by each transaction in order to release the locks when the transaction commits or rolls back, and also track the locks that a transaction is relying on to make progress in order to detect deadlocks. A transaction likely holds many locks, so we use a hash set (std::unordered_set) to store the locks held by each transaction. On the other hand, we expect the number of locks that a transaction is waiting on to be small, so we use a std::flat_set to store the locks that a transaction is waiting on.

Figure 4: Relation and Group Classes

When a transaction suspends, the deadlock detector (Figure 5) is triggered by the Database (Figure 6) to check the waits-for graph for cycles. We choose to make this a separate class to ensure modularity (we can change the deadlock detection logic without affecting the Database class). The Database class is also the class that handles the aborting of transactions, and resumption of transactions in the event a deadlock is detected.

Figure 5: Deadlock Detector Class

Figure 6: Database Class

iii

We first describe the functions in the Lock classes:

`Lock::acquire(tx_id: TID, mode: LockMode) -> boolean`

`Lock::acquire` attempts to acquire the lock for the transaction with the given ID in the specified mode (shared or exclusive). It returns true if the lock was successfully acquired, and false otherwise.

For SLocks, it just adds `tx_id` to the set of lock holders if it is not already in there. For XSLocks, to check if the lock can be acquired, as in Algorithm 1.

Algorithm 1: XSLock::Acquire

```
procedure XSLock::Acquire(tx_id: TID, mode: LockMode) → boolean
    if mode = EXCLUSIVE then
        if holder_count() >= 2 then
            return false
        else if holder_count() = 1 and ¬ is_held_by(tx_id) then
            return false
```

```
    else
        # Upgrade or Acquire
        lock_holders.insert(tx_id)
        held_exclusively ← true
        return true
    end if
  else
    if held_exclusively then
        return is_held_by(tx_id)
    else
        lock_holders.insert(tx_id)
        return true
    end if
  end if
end procedure
```

`Lock::release(tx_id: TID) -> boolean`

`Lock::release` removes the relevant transaction from the set of lock holders. It should only be called when the transaction is actually holding the lock.

`Lock::current_holders() -> &Set<TID>`

`Lock::current_holders` directly returns a reference to the set of transaction IDs currently holding the lock. This is used by the deadlock detector to construct the waits-for graph.

Most of the time, apart from registering the transaction with the lock, we also need to register the lock with the transaction. Thus, we provide the method `Transaction::acquire(lock: Lock, mode: LockMode) -> boolean` that attempts to acquire the lock for the transaction, and if successful, also adds the lock to the set of locks held by the transaction. If the lock cannot be acquired, then the transaction is added to the set of transactions waiting on the lock, and the method returns false.

When a transaction runs until it either completes its operation or suspends, the call stack will unwind until control is returned to the relevant method within the Database class. We will then call `Database::on_control` which deal with cycle detection and fixing.

Algorithm 2: Database::OnControl

```
procedure Database::OnControl(tid: TID, status: StatusCode)
  if status = SUSPENDED then
    while true do
        victim ← deadlock_detector.detect_cycle(tid)
        if victim is empty then break
        # Deadlock detected: rollback a victim
        rollback_transaction(victim)
        if victim ≠ tid then
            # If we didn't abort ourselves, we can try to resume
            resume_transaction(tid)
            return
```

```
        end if
    end while
    print "Transaction " + tid + " was suspended."
else
    transactions[tid].clear_required_locks()
end if
end procedure
```

The key step in cycle detection is done via `DeadlockDetector::detect_cycle(tid: TID) -> Optional<TID>`, which performs a depth first search starting from the given transaction ID, traversing through the waits-for graph by looking at the locks that the transaction is waiting on, and then looking at the transactions that are holding those locks, and so on. If we run into any back edge, then we have found a cycle. We then locate the youngest transaction in the cycle and return it as the victim to be rolled back. This is described in more detail in Algorithm 3.

Algorithm 3: Deadlock Detection

```
procedure DeadlockDetector::DetectCycle(start_tid: TID) → Optional<TID>
    visited_tx.clear()
    cycle.clear()
    if DFS(start_tid) then
        # Locate the youngest transaction in the cycle to abort
        victim ← cycle[0]
        for each tid in cycle do
            if transactions[tid].born_at > transactions[victim].born_at then
                victim ← tid
            end if
        end for
        return victim
    end if
    return empty
end procedure
procedure DeadlockDetector::DFS(tid: TID) → boolean
    if visited_tx[tid] = VISITING then return true
    if visited_tx[tid] = VISITED then return false
    visited_tx[tid] ← VISITING
    for each lock in transactions[tid].required_locks do
        for each holding_tid in lock.current_holders() do
            if holding_tid = tid then continue
            if DFS(holding_tid) then
                cycle.push(tid)
                return true
            end if
        end for
    end for
```

```
    visited_tx[tid] ← VISITED
    return false
end procedure
```

b

i

The actual tuples are stored in the Relation class's tuples vector. We use a StableVector that works as a linked list of fixed-size blocks. This allows us to have stable pointers to tuples that can be stored in the various indices. Using a normal std::vector would require us to update all the pointers in the indices every time we resize the vector, while using a linked list would result in worse cache performance and allocation overhead.

We index

2

3

a

	Action	Time (ms)	# of Answers
Step 1	Import:	23308	
	Rollback:	5145	
Step 2	Import:	22480	
	Commit:	3363	
Step 3	Query 1:	0	4
	Query 2:	1735	264
	Query 3:	0	6
	Query 4:	0	34
	Query 5:	0	719
	Query 6:	224	1048532
	Query 7:	0	67
	Query 8:	8	7790
	Query 9:	1150	27247
	Query 10:	0	4
	Query 11:	0	224
	Query 12:	0	15
	Query 13:	0	472
	Query 14:	112	795970
	Commit:	42	
Step 4	Delete:	811	
	Rollback:	247	
Step 5	Delete:	3509	

	Action	Time (ms)	# of Answers
	Commit:	358	
Step 6	Query 1:	1	0
	Query 2:	139	0
	Query 3:	0	0
	Query 4:	0	0
	Query 5:	0	0
	Query 6:	198	923871
	Query 7:	0	0
	Query 8:	13	4040
	Query 9:	846	22522
	Query 10:	0	0
	Query 11:	0	186
	Query 12:	0	9
	Query 13:	2	413
	Query 14:	103	701364
	Commit:	25	

Table 1: Benchmarking Results

b